

```
!pip install -q opencv-python numpy matplotlib pandas
```

```
# =====
# HARRIS CORNER DETECTION ANALYSIS
# Detection and Analysis of Corners in Images with
# Varying Textures and Patterns
# =====

# =====
# 1. IMPORT LIBRARIES
# =====

import cv2
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from google.colab import files
from IPython.display import display

print("=" * 90)
print("                HARRIS CORNER DETECTION ANALYSIS")
print("=" * 90)

# =====
# 2. UPLOAD IMAGES
# =====

print("\nUpload 5 images with different textures/patterns.")
print("Example: Brick Wall, Checkerboard, Building, Leaves, PCB Board")

uploaded = files.upload()

image_files = list(uploaded.keys())

if len(image_files) == 0:
    raise ValueError("No images uploaded.")

print("\nUploaded Images:")
for i, name in enumerate(image_files):
    print(f"{i+1}. {name}")

# =====
# 3. PARAMETERS
# =====

BLOCK_SIZE = 2
APERTURE_SIZE = 3
HARRIS_K = 0.04

THRESHOLD_RATIO = 0.01

BORDER_TYPE = cv2.BORDER_DEFAULT

# =====
# 4. HARRIS CORNER DETECTION FUNCTION
# =====

def detect_harris_corners(
    image,
    block_size=BLOCK_SIZE,
    aperture_size=APERTURE_SIZE,
    k=HARRIS_K,
    threshold_ratio=THRESHOLD_RATIO
):
    # Convert image to grayscale
    gray = cv2.cvtColor(
        image,
        cv2.COLOR_BGR2GRAY
    )
```

```

# Convert to float32
gray_float = np.float32(gray)

# Harris response
response = cv2.cornerHarris(
    gray_float,
    block_size,
    aperture_size,
    k
)

# Dilate response for visualization
response_dilated = cv2.dilate(
    response,
    None
)

# Threshold
threshold = (
    threshold_ratio *
    response.max()
)

# Initial corner mask
corner_mask = (
    response > threshold
).astype(np.uint8)

# =====
# NON-MAXIMUM SUPPRESSION
# =====

kernel = np.ones(
    (3, 3),
    np.uint8
)

local_max = (
    response ==
    cv2.dilate(
        response,
        kernel
    )
)

final_mask = (
    local_max &
    (response > threshold)
)

# =====
# REMOVE BORDER CORNERS
# =====

margin = 5

final_mask[
    :margin,
    :
] = False

final_mask[
    -margin:,
    :
] = False

final_mask[
    :,
    :margin
] = False

final_mask[
    :,
    -margin:
] = False

```

```

# =====
# GET CORNER COORDINATES
# =====

ys, xs = np.where(
    final_mask
)

corners = list(
    zip(xs, ys)
)

# =====
# VISUALIZATION IMAGE
# =====

visualization = image.copy()

for x, y in corners:

    cv2.circle(
        visualization,
        (int(x), int(y)),
        4,
        (0, 0, 255),
        -1
    )

# =====
# CORNER DENSITY
# =====

total_pixels = (
    gray.shape[0] *
    gray.shape[1]
)

corner_density = (
    len(corners) /
    total_pixels
) * 100

return {
    "gray": gray,
    "response": response,
    "response_dilated": response_dilated,
    "threshold": threshold,
    "corners": corners,
    "visualization": visualization,
    "corner_density": corner_density
}

# =====
# 5. PROCESS ALL IMAGES
# =====

results = []

for image_file in image_files:

    image = cv2.imread(
        image_file
    )

    if image is None:
        print(
            f"Could not read {image_file}"
        )
        continue

    result = detect_harris_corners(
        image
    )

```

```

    )

    result["filename"] = image_file
    result["image"] = image

    results.append(
        result
    )

# =====
# 6. IMAGE LABELS
# =====

default_labels = [
    "Brick Wall",
    "Checkerboard",
    "Building",
    "Leaves",
    "PCB Board"
]

labels = []

for i in range(len(results)):

    if i < len(default_labels):

        labels.append(
            default_labels[i]
        )

    else:

        labels.append(
            f"Image {i+1}"
        )

# =====
# 7. PRINT DETECTION SUMMARY
# =====

print("\n")
print("=" * 90)
print("                                HARRIS CORNER RESULTS")
print("=" * 90)

for i, result in enumerate(results):

    print(
        f"\n{labels[i]}"
    )

    print(
        f"Image Size      : "
        f"{result['gray'].shape[1]} x "
        f"{result['gray'].shape[0]}"
    )

    print(
        f"Total Pixels      : "
        f"{result['gray'].size:,}"
    )

    print(
        f"Harris Response : "
        f"{result['response'].max():.4e}"
    )

    print(
        f"Threshold         : "
        f"{result['threshold']:.4e}"
    )

    print(
        f"Corners Detected: "

```

```

        f"{len(result['corners']):,}"
    )

    print(
        f"Corner Density : "
        f"{result['corner_density']:.2f}%"
    )

# =====
# 8. DISPLAY ORIGINAL IMAGES
# =====

n = len(results)

plt.figure(
    figsize=(4*n, 5)
)

for i, result in enumerate(results):

    plt.subplot(
        1,
        n,
        i+1
    )

    rgb = cv2.cvtColor(
        result["image"],
        cv2.COLOR_BGR2RGB
    )

    plt.imshow(
        rgb
    )

    plt.title(
        f"Image {i+1}\n{labels[i]}",
        fontsize=12,
        fontweight="bold"
    )

    plt.axis("off")

plt.suptitle(
    "Input Images with Varying Textures and Patterns",
    fontsize=18,
    fontweight="bold"
)

plt.tight_layout()
plt.show()

# =====
# 9. DISPLAY GRAYSCALE IMAGES
# =====

plt.figure(
    figsize=(4*n, 5)
)

for i, result in enumerate(results):

    plt.subplot(
        1,
        n,
        i+1
    )

    plt.imshow(
        result["gray"],
        cmap="gray"
    )

    plt.title(

```

```

        f"{labels[i]}\nGrayscale",
        fontsize=12,
        fontweight="bold"
    )

    plt.axis("off")

plt.suptitle(
    "Grayscale Images",
    fontsize=18,
    fontweight="bold"
)

plt.tight_layout()
plt.show()

# =====
# 10. DISPLAY HARRIS RESPONSE
# =====

plt.figure(
    figsize=(4*n, 5)
)

for i, result in enumerate(results):

    plt.subplot(
        1,
        n,
        i+1
    )

    plt.imshow(
        result["response"],
        cmap="jet"
    )

    plt.title(
        f"{labels[i]}\nHarris Response",
        fontsize=11,
        fontweight="bold"
    )

    plt.axis("off")

plt.suptitle(
    "Harris Corner Response",
    fontsize=18,
    fontweight="bold"
)

plt.tight_layout()
plt.show()

# =====
# 11. DISPLAY DETECTED CORNERS
# =====

plt.figure(
    figsize=(4*n, 5)
)

for i, result in enumerate(results):

    plt.subplot(
        1,
        n,
        i+1
    )

    rgb = cv2.cvtColor(
        result["visualization"],
        cv2.COLOR_BGR2RGB
    )

```

```

plt.imshow(
    rgb
)

plt.title(
    f"{labels[i]}\n"
    f"Corners = {len(result['corners']):,}",
    fontsize=11,
    fontweight="bold"
)

plt.axis("off")

plt.suptitle(
    "Harris Corners Detected",
    fontsize=18,
    fontweight="bold"
)

plt.tight_layout()
plt.show()

# =====
# 12. ZOOMED CORNER VIEW
# =====

plt.figure(
    figsize=(4*n, 5)
)

for i, result in enumerate(results):

    plt.subplot(
        1,
        n,
        i+1
    )

    rgb = cv2.cvtColor(
        result["visualization"],
        cv2.COLOR_BGR2RGB
    )

    h, w = rgb.shape[:2]

    # Central region
    y1 = int(h * 0.25)
    y2 = int(h * 0.75)

    x1 = int(w * 0.25)
    x2 = int(w * 0.75)

    crop = rgb[
        y1:y2,
        x1:x2
    ]

    plt.imshow(
        crop
    )

    plt.title(
        f"{labels[i]}\nZoomed View",
        fontsize=11,
        fontweight="bold"
    )

    plt.axis("off")

plt.suptitle(
    "Zoomed View of Detected Corners",
    fontsize=18,
    fontweight="bold"
)

```

```

)

plt.tight_layout()
plt.show()

# =====
# 13. CORNER COORDINATE DISTRIBUTION
# =====

plt.figure(
    figsize=(5*n, 5)
)

for i, result in enumerate(results):

    plt.subplot(
        1,
        n,
        i+1
    )

    corners = result["corners"]

    if len(corners) > 0:

        xs = [
            point[0]
            for point in corners
        ]

        ys = [
            point[1]
            for point in corners
        ]

        plt.scatter(
            xs,
            ys,
            s=8
        )

    plt.gca().invert_yaxis()

    plt.title(
        labels[i],
        fontsize=11,
        fontweight="bold"
    )

    plt.xlabel(
        "X"
    )

    plt.ylabel(
        "Y"
    )

    plt.grid(
        alpha=0.3
    )

plt.suptitle(
    "Spatial Distribution of Harris Corners",
    fontsize=18,
    fontweight="bold"
)

plt.tight_layout()
plt.show()

# =====
# 14. QUANTITATIVE COMPARISON TABLE
# =====

table_data = []

```

```

table_data = []

for i, result in enumerate(results):

    image = result["image"]

    h, w = result["gray"].shape

    total_pixels = h * w

    response_max = result[
        "response"
    ].max()

    threshold = result[
        "threshold"
    ]

    corners_detected = len(
        result["corners"]
    )

    density = result[
        "corner_density"
    ]

    table_data.append({

        "Image": labels[i],

        "Image Size": f"{w} × {h}",

        "Total Pixels": total_pixels,

        "Response R (Max)": response_max,

        "Threshold (0.01 × Rmax)": threshold,

        "Corners Detected": corners_detected,

        "Corner Density (%)": density

    })

comparison_df = pd.DataFrame(
    table_data
)

comparison_df[
    "Response R (Max)"
] = comparison_df[
    "Response R (Max)"
].map(
    lambda x: f"{x:.2e}"
)

comparison_df[
    "Threshold (0.01 × Rmax)"
] = comparison_df[
    "Threshold (0.01 × Rmax)"
].map(
    lambda x: f"{x:.2e}"
)

comparison_df[
    "Corner Density (%)"
] = comparison_df[
    "Corner Density (%)"
].round(2)

print("\n")

```

```

print("=" * 110)
print("                                QUANTITATIVE ANALYSIS")
print("=" * 110)

display(
    comparison_df
)

# =====
# 15. CORNER COUNT GRAPH
# =====

corner_counts = [
    len(result["corners"])
    for result in results
]

plt.figure(
    figsize=(12, 7)
)

bars = plt.bar(
    labels,
    corner_counts,
    color=[
        "firebrick",
        "darkorange",
        "seagreen",
        "royalblue",
        "purple"
    ],
    edgecolor="black"
)

plt.title(
    "Number of Corners Detected",
    fontsize=20,
    fontweight="bold"
)

plt.xlabel(
    "Image",
    fontsize=13
)

plt.ylabel(
    "No. of Corners",
    fontsize=13
)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.4
)

for bar, value in zip(
    bars,
    corner_counts
):
    plt.text(
        bar.get_x() +
        bar.get_width()/2,
        bar.get_height() +
        max(corner_counts) * 0.02,
        f"{value:}",
        ha="center",
        va="bottom",
        fontsize=12,
        fontweight="bold"
    )

```

```

plt.tight_layout()
plt.show()

# =====
# 16. CORNER DENSITY GRAPH
# =====

densities = [
    result["corner_density"]
    for result in results
]

plt.figure(
    figsize=(12, 7)
)

bars = plt.bar(
    labels,
    densities,
    color=[
        "firebrick",
        "darkorange",
        "seagreen",
        "royalblue",
        "purple"
    ],
    edgecolor="black"
)

plt.title(
    "Corner Density Comparison",
    fontsize=20,
    fontweight="bold"
)

plt.xlabel(
    "Image",
    fontsize=13
)

plt.ylabel(
    "Corner Density (%)",
    fontsize=13
)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.4
)

for bar, value in zip(
    bars,
    densities
):
    plt.text(
        bar.get_x() +
        bar.get_width()/2,
        bar.get_height() +
        max(densities) * 0.02,
        f"{value:.2f}%",
        ha="center",
        va="bottom",
        fontsize=11,
        fontweight="bold"
    )

```

```

plt.tight_layout()
plt.show()

# =====
# 17. HARRIS RESPONSE GRAPH
# =====

responses = [
    result["response"].max()
    for result in results
]

plt.figure(
    figsize=(12, 7)
)

plt.bar(
    labels,
    responses,
    edgecolor="black"
)

plt.title(
    "Maximum Harris Response",
    fontsize=20,
    fontweight="bold"
)

plt.xlabel(
    "Image",
    fontsize=13
)

plt.ylabel(
    "Maximum Harris Response",
    fontsize=13
)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.4
)

plt.tight_layout()
plt.show()

# =====
# 18. PARAMETER SENSITIVITY EXPERIMENT
# =====

# Use first uploaded image for parameter analysis

test_image = results[0]["image"]

gray_test = cv2.cvtColor(
    test_image,
    cv2.COLOR_BGR2GRAY
)

gray_test_float = np.float32(
    gray_test
)

block_sizes = [
    2,
    3,
    5

```

```

]

k_values = [
    0.04,
    0.05,
    0.06
]

sensitivity_results = []

for block in block_sizes:
    for k_value in k_values:
        response = cv2.cornerHarris(
            gray_test_float,
            block,
            APERTURE_SIZE,
            k_value
        )

        threshold = (
            THRESHOLD_RATIO *
            response.max()
        )

        mask = (
            response >
            threshold
        ).astype(
            np.uint8
        )

        # Non-maximum suppression

        local_max = (
            response ==
            cv2.dilate(
                response,
                np.ones(
                    (3, 3),
                    np.uint8
                )
            )
        )

        final = (
            local_max &
            (response > threshold)
        )

        count = np.count_nonzero(
            final
        )

        sensitivity_results.append({
            "Block Size": block,
            "Harris k": k_value,
            "Threshold": threshold,
            "Corners Detected": count
        })

sensitivity_df = pd.DataFrame(
    sensitivity_results
)

print("\n")
print("=" * 100)
print("PARAMETER SENSITIVITY ANALYSIS")

```

```

print("=" * 100)

display(
    sensitivity_df
)

# =====
# 19. PARAMETER SENSITIVITY GRAPH
# =====

plt.figure(
    figsize=(11, 7)
)

for k_value in k_values:

    subset = sensitivity_df[
        sensitivity_df[
            "Harris k"
        ] == k_value
    ]

    plt.plot(
        subset["Block Size"],
        subset["Corners Detected"],
        marker="o",
        linewidth=2,
        label=f"k = {k_value}"
    )

plt.title(
    "Effect of Harris Parameters on Corner Detection",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel(
    "Block Size",
    fontsize=13
)

plt.ylabel(
    "Corners Detected",
    fontsize=13
)

plt.xticks(
    block_sizes
)

plt.legend()

plt.grid(
    True,
    linestyle="--",
    alpha=0.4
)

plt.tight_layout()
plt.show()

# =====
# 20. FIND HIGHEST CORNER IMAGE
# =====

max_index = np.argmax(
    corner_counts
)

min_index = np.argmin(
    corner_counts
)

```

```

highest_image = labels[
    max_index
]

lowest_image = labels[
    min_index
]

highest_count = corner_counts[
    max_index
]

lowest_count = corner_counts[
    min_index
]

# =====
# 21. OBSERVATIONS
# =====

print("\n")
print("=" * 90)
print("                                OBSERVATIONS")
print("=" * 90)

print(
    "\n• Harris corner detection identifies points where "
    "intensity changes significantly in multiple directions."
)

print(
    "• Images containing repeated patterns and rich textures "
    "generally produce more Harris corners."
)

print(
    "• Smooth regions usually produce very few or no corners."
)

print(
    "• Corners tend to be concentrated around junctions, "
    "edges, textured regions and pattern intersections."
)

print(
    "• The number of detected corners depends on the image "
    "content and threshold value."
)

print(
    "• The Harris detector is invariant to image rotation "
    "in its basic gradient-based formulation."
)

print(
    f"• In this experiment, {highest_image} produced the "
    f"highest number of detected corners "
    f"({highest_count:,})."
)

print(
    f"• {lowest_image} produced the lowest number of "
    f"detected corners ({lowest_count:,})."
)

# =====
# 22. IMAGE-SPECIFIC ANALYSIS
# =====

print("\n")
print("=" * 90)
print("                                IMAGE-WISE ANALYSIS")
print("=" * 90)

```

```

print(
    30)

for i, result in enumerate(results):

    count = len(
        result["corners"]
    )

    density = result[
        "corner_density"
    ]

    print(
        f"\n{labels[i]}:"
    )

    if density < 0.20:

        print(
            "Low corner density – image contains relatively "
            "smooth or structured regions."
        )

    elif density < 0.50:

        print(
            "Moderate corner density – image contains a "
            "combination of edges, patterns and textured regions."
        )

    else:

        print(
            "High corner density – image contains rich texture, "
            "repetitive patterns or many intensity junctions."
        )

    print(
        f"Corners detected: {count:},"
    )

    print(
        f"Corner density: {density:.2f}%"
    )

# =====
# 23. THEORETICAL CONCEPTS
# =====

print("\n")
print("=" * 90)
print("                THEORETICAL CONCEPTS")
print("=" * 90)

print(
    "\nWHAT IS A CORNER?"
)

print(
    "A corner is a point in an image where intensity changes "
    "significantly in multiple directions."
)

print(
    "\nHARRIS CORNER DETECTION"
)

print(
    "1. Compute image gradients Ix and Iy."
)

print(
    "2. Compute products of derivatives."
)

```

```

print(
    "3. Apply a Gaussian window to obtain the structure tensor."
)

print(
    "4. Compute the Harris response."
)

print(
    "5. Apply a threshold to identify strong corners."
)

print(
    "\nHARRIS RESPONSE FORMULA:"
)

print(
    "R = det(M) - k × (trace(M))²"
)

print(
    "where M is the structure tensor and k is the Harris "
    "empirical constant."
)

print(
    "\nEIGENVALUE INTERPRETATION:"
)

print(
    "• Both eigenvalues large → Corner"
)

print(
    "• One large and one small → Edge"
)

print(
    "• Both small → Flat region"
)

# =====
# 24. SIGNIFICANCE OF CORNERS
# =====

print("\n")
print("=" * 90)
print("                SIGNIFICANCE OF CORNERS")
print("=" * 90)

print(
    "\n• Corners provide important information about the "
    "structure of an image."
)

print(
    "• They are relatively stable under illumination changes "
    "and small transformations."
)

print(
    "• Corners are useful as feature points for computer vision."
)

print(
    "• They improve feature matching and image registration."
)

print(
    "• They can be used in object recognition."
)

```

```

print(
    "• They are useful for motion tracking and SLAM."
)

print(
    "• They can support 3D reconstruction."
)

# =====
# 25. APPLICATIONS
# =====

print("\n")
print("=" * 90)
print("                APPLICATIONS")
print("=" * 90)

print(
    "\n• Feature Matching"
)

print(
    "• Image Registration"
)

print(
    "• 3D Reconstruction"
)

print(
    "• SLAM (Simultaneous Localization and Mapping)"
)

print(
    "• Object Recognition"
)

print(
    "• Motion Tracking"
)

print(
    "• Augmented Reality"
)

# =====
# 26. CONCLUSION
# =====

print("\n")
print("=" * 90)
print("                CONCLUSION")
print("=" * 90)

print(
    "\nHarris corner detection is an effective technique for "
    "identifying important feature points in images."
)

print(
    "The experiment demonstrates that the number and distribution "
    "of detected corners depend strongly on image texture, "
    "patterns and structural content."
)

print(
    "Images containing repeated patterns and rich textures "
    "generally produce more corners, whereas smooth regions "
    "produce fewer corners."
)

print(
    "The Harris detector provides meaningful and repeatable "
    "corner points that can be used for feature matching."
)

```

```

"image registration, object recognition, tracking and "
"other computer vision applications."
)

# =====
# 27. OVERALL RESULT
# =====

print("\n")
print("=" * 90)
print("                                OVERALL RESULT")
print("=" * 90)

print(
    "\nHarris corner detection provides accurate, stable and "
    "meaningful feature points."
)

print(
    "It performs particularly well on images containing "
    "rich textures, repeated patterns and structural junctions."
)

print(
    "The selection of block size, Harris k and threshold "
    "significantly influences the number of detected corners."
)

# =====
# 28. SAVE RESULTS
# =====

comparison_df.to_csv(
    "harris_corner_comparison.csv",
    index=False
)

sensitivity_df.to_csv(
    "harris_parameter_analysis.csv",
    index=False
)

# =====
# 29. SAVE DETECTED CORNER IMAGES
# =====

for i, result in enumerate(results):

    filename = (
        f"harris_corners_{i+1}.png"
    )

    cv2.imwrite(
        filename,
        result["visualization"]
    )

# =====
# 30. FINAL MESSAGE
# =====

print("\n")
print("=" * 90)
print("                                HARRIS CORNER EXPERIMENT COMPLETED")
print("=" * 90)

print("\nGenerated outputs:")

print("✓ Input Images")
print("✓ Grayscale Images")
print("✓ Harris Response Maps")

```

```
print("✓ Detected Harris Corners")
print("✓ Zoomed Corner Views")
print("✓ Corner Distribution Plots")
print("✓ Quantitative Comparison Table")
print("✓ Corner Count Graph")
print("✓ Corner Density Graph")
print("✓ Harris Response Graph")
print("✓ Parameter Sensitivity Analysis")
print("✓ Parameter Sensitivity Graph")
print("✓ Image-wise Analysis")
print("✓ Observations")
print("✓ Theoretical Concepts")
print("✓ Applications")
print("✓ Significance of Corners")
print("✓ Conclusion")
print("✓ CSV Result Files")

print("\n" + "=" * 90)
```


=====

HARRIS CORNER DETECTION ANALYSIS

=====

Upload 5 images with different textures/patterns.

Example: Brick Wall, Checkerboard, Building, Leaves, PCB Board

IMAGE.jpg

IMAGE.jpg(image/jpeg) - 15042 bytes, last modified: 7/26/2026 - 100% done

Saving IMAGE.jpg to IMAGE (3).jpg

Uploaded Images:

1. IMAGE (3).jpg

=====

HARRIS CORNER RESULTS

=====

Brick Wall

Image Size : 300 x 168

Total Pixels : 50,400

Harris Response : 9.5050e+07

Threshold : 9.5050e+05

Corners Detected: 822

Corner Density : 1.63%

Input Images with Varying Textures and Patterns

Image 1
Brick Wall



Grayscale Images

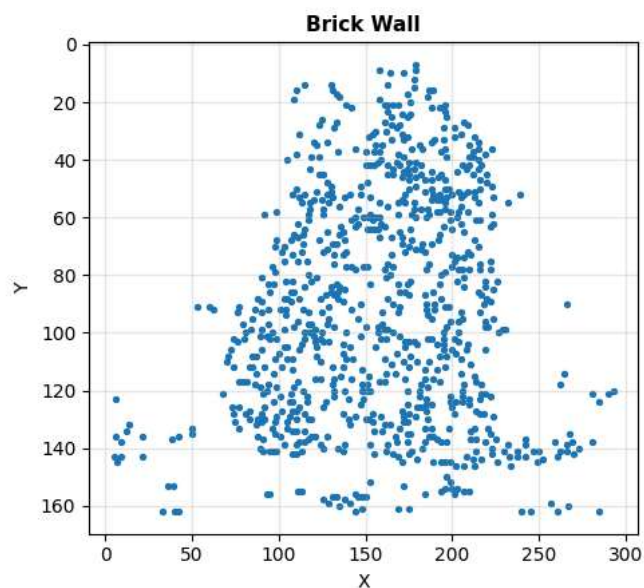
Brick Wall
Grayscale



Harris Corner Response

Brick Wall
Harris Response



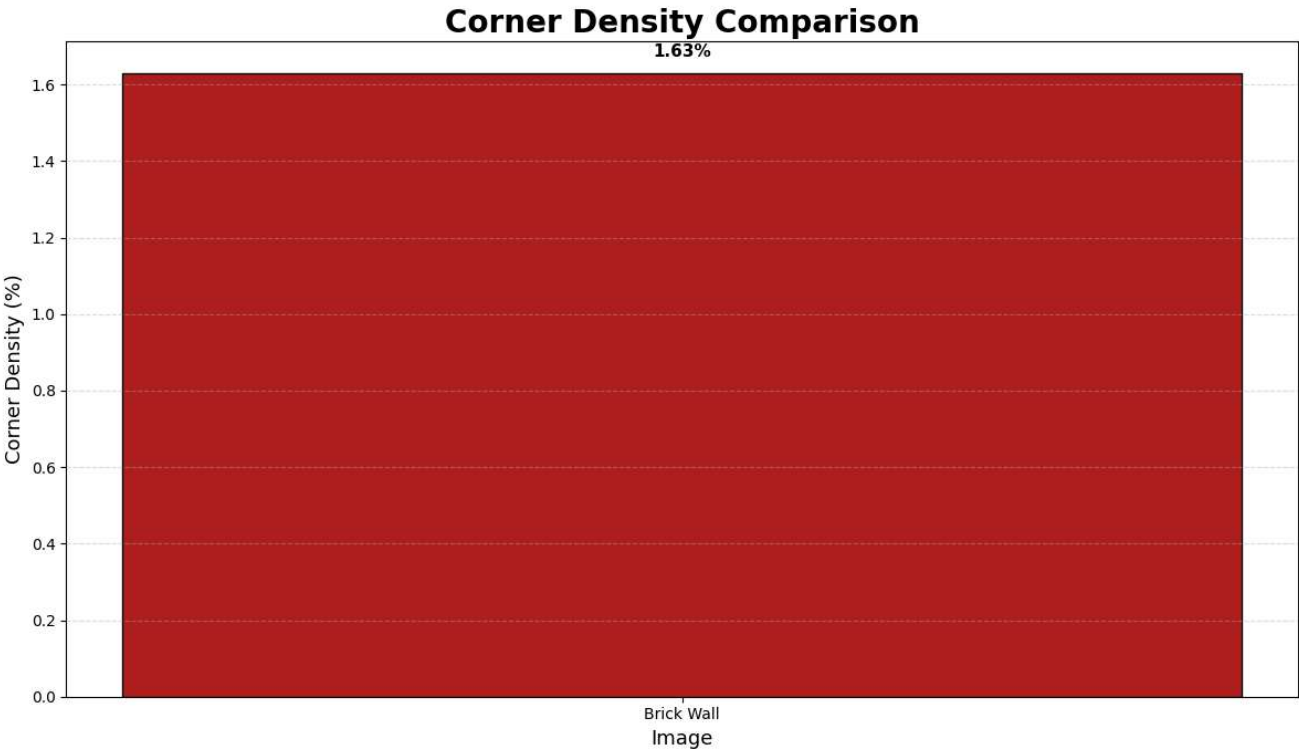
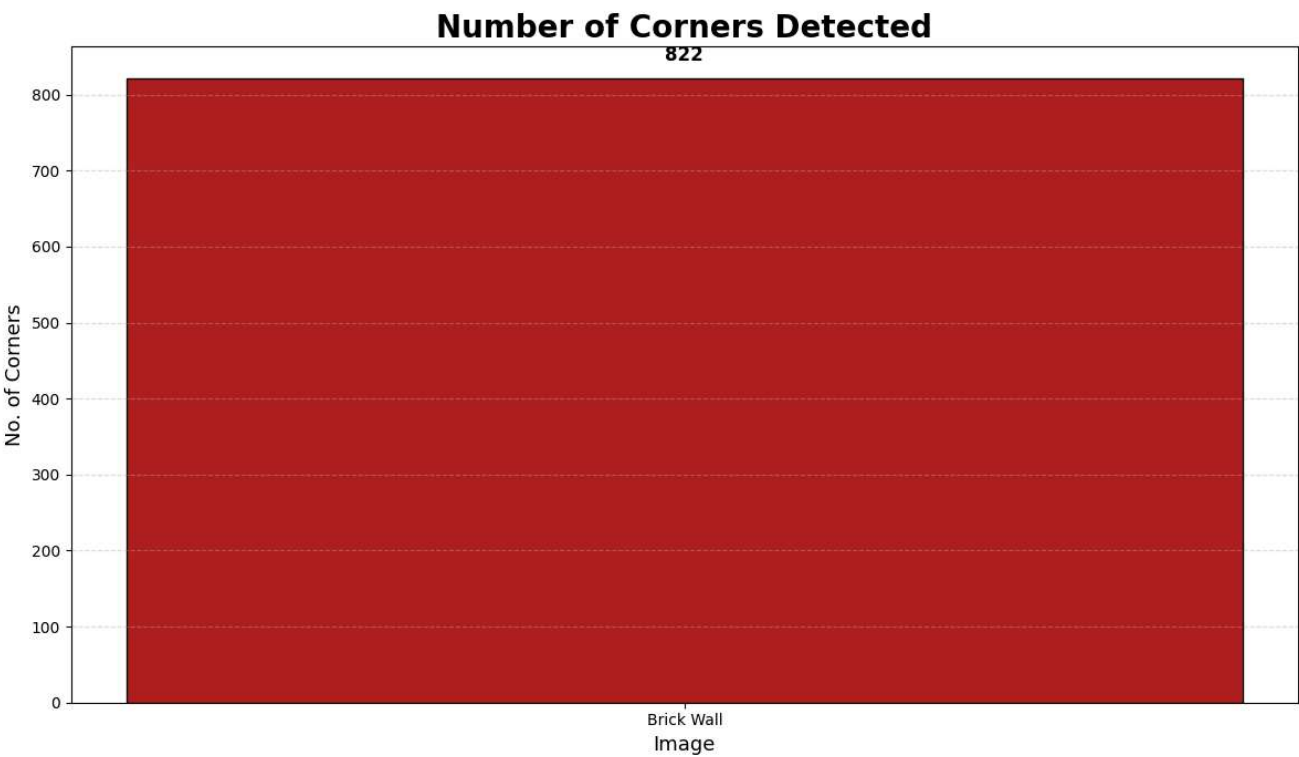
**Harris Corners Detected****Brick Wall
Corners = 822****Zoomed View of Detected Corners****Brick Wall
Zoomed View****Spatial Distribution of Harris Corners**

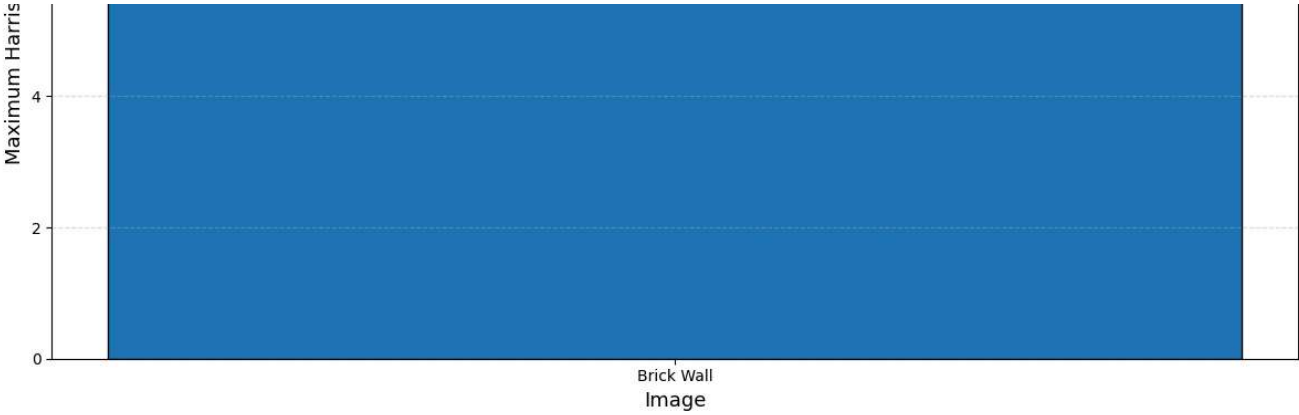
=====

QUANTITATIVE ANALYSIS

=====

	Image	Image Size	Total Pixels	Response R (Max)	Threshold ($0.01 \times R_{max}$)	Corners Detected	Corner Density (%)	
0	Brick Wall	300 × 168	50400	9.50e+07	9.50e+05	822	1.63	



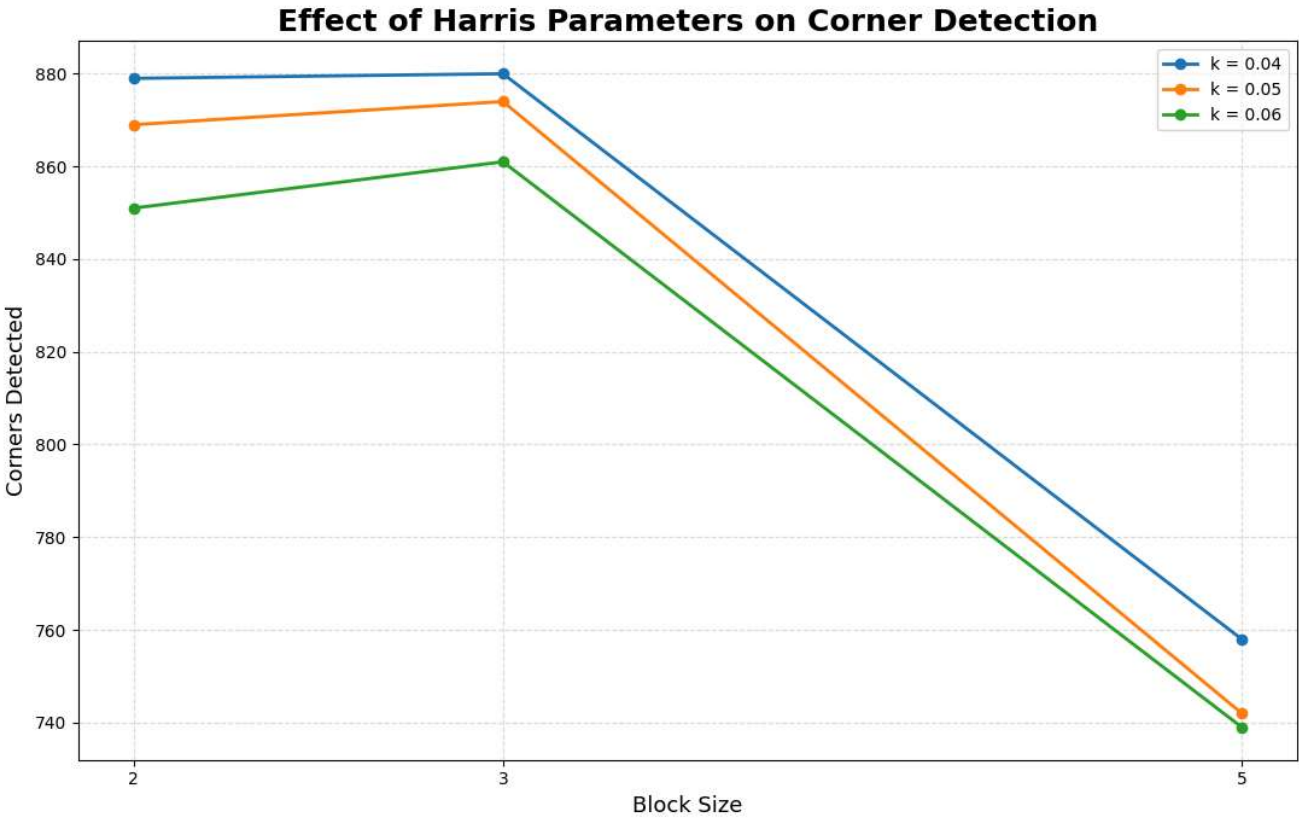


=====

PARAMETER SENSITIVITY ANALYSIS

=====

	Block Size	Harris k	Threshold	Corners Detected	
0	2	0.04	950499.12500	879	
1	2	0.05	890101.25000	869	
2	2	0.06	829703.50000	851	
3	3	0.04	651611.25000	880	
4	3	0.05	613072.31250	874	
5	3	0.06	574533.43750	861	
6	5	0.04	313207.59375	758	
7	5	0.05	295988.78125	742	
8	5	0.06	278770.03125	739	



=====

OBSERVATIONS

=====

- Harris corner detection identifies points where intensity changes significantly in multiple directions.
- Images containing repeated patterns and rich textures generally produce more Harris corners.

- Smooth regions usually produce very few or no corners.
- Corners tend to be concentrated around junctions, edges, textured regions and pattern intersections.
- The number of detected corners depends on the image content and threshold value.
- The Harris detector is invariant to image rotation in its basic gradient-based formulation.
- In this experiment, Brick Wall produced the highest number of detected corners (822).
- Brick Wall produced the lowest number of detected corners (822).

=====

IMAGE-WISE ANALYSIS

=====

Brick Wall:

High corner density – image contains rich texture, repetitive patterns or many intensity junctions.

Corners detected: 822

Corner density: 1.63%